

Understanding and Improving Proximity Graph Based Maximum Inner Product Search

Jie Liu,^{*} Xiao Yan,^{*} Xinyan Dai, Zhirong Li, James Cheng, Ming-Chang Yang

The Chinese University of Hong Kong
{jliu, xyan, xydai, zrli6, jcheng, mcyang}@cse.cuhk.edu.hk

Abstract

The inner-product navigable small world graph (ip-NSW) represents the state-of-the-art method for approximate maximum inner product search (MIPS) and it can achieve an order of magnitude speedup over the fastest baseline. However, to date it is still unclear where its exceptional performance comes from. In this paper, we show that there is a strong norm bias in the MIPS problem, which means that the large norm items are very likely to become the result of MIPS. Then we explain the good performance of ip-NSW as matching the norm bias of the MIPS problem — large norm items have big in-degrees in the ip-NSW proximity graph and a walk on the graph spends the majority of computation on these items, thus effectively avoids unnecessary computation on small norm items. Furthermore, we propose the ip-NSW+ algorithm, which improves ip-NSW by introducing an additional angular proximity graph. Search is first conducted on the angular graph to find the angular neighbors of a query and then the MIPS neighbors of these angular neighbors are used to initialize the candidate pool for search on the inner-product proximity graph. Experiment results show that ip-NSW+ consistently and significantly outperforms ip-NSW and provides more robust performance under different data distributions.

1 Introduction

For a query q , maximum inner product search (MIPS) finds an item that maximizes $q^\top x_i$ in a dataset $\mathcal{X} = \{x_i \in \mathbb{R}^d | i = 1, \dots, n\}$ containing n items (Ram and Gray 2012). MIPS has a number of applications in recommender systems, computer vision and machine learning. Examples include recommendation based on user and item embeddings learned via matrix factorization (Koren, Bell, and Volinsky 2009), object matching with visual descriptor (Felzenszwalb et al. 2010), memory network training (Chandar et al. 2016) and reinforcement learning (Jun et al. 2017). In practice, it is usually required to find the top- k items having the largest inner product with q . When the dataset is large and the dimension (i.e., d) is high, exact MIPS is usually too costly

and finding approximate MIPS (i.e., items with inner product close to the maximum) suffices for most applications. Therefore, we focus on approximate MIPS in this paper.

Related work. Due to its broad range of applications, many algorithms for MIPS have been proposed. Tree-based methods such as cone tree (Ram and Gray 2012) and PCA tree (Bachrach et al. 2014) were first used but they suffer from poor performance on high dimensional datasets. Locality sensitive hashing (LSH) based methods are widely used for similarity search (Li et al. 2018). ALSH (Shrivastava and Li 2014), Simple-LSH (Neyshabur and Srebro 2015) and Norm-Range LSH (Yan et al. 2018), transform MIPS into Euclidean or angular similarity search and reuse existing hash functions. LEMP (Teflioudi, Gemulla, and Mykytiuk 2015) and FEXIPRO (Li et al. 2017) target exact MIPS and adopt various pruning rules to avoid unnecessary computation. Maximus (Abuzaid et al. 2019) shows that the pruning-based methods do not always outperform brute-force linear scans using optimized computation libraries.

ip-NSW. In a proximity graph, each item is connected to some items that are most similar to it w.r.t. a given similarity function (Hajebi et al. 2011). A similarity search query is processed by a walk in the graph, which keeps moving towards items that are most similar to the query. Proximity graph based methods achieve excellent recall-time performance¹ for Euclidean distance nearest neighbor search (Euclidean NNS) and an number of variants have been proposed (Fu et al. 2019; Harwood and Drummond 2016; Wang et al. 2013). Among them, the navigable small word graph (NSW) (Malkov et al. 2014) and its hierarchical version (HNSW) (Malkov and Yashunin 2018) represent the state-of-the-art and we introduce NSW in greater details in Section 3. Morozov and Babenko (2018) showed that NSW also works well for MIPS. They proposed the ip-NSW algorithm, which directly uses inner product as similarity function to construct and search NSW. ip-NSW outperforms all existing MIPS algorithms (including those mentioned in the related work) by a large margin in terms of recall-time performance and the speedup can be an order of magnitude for achieving the same recall (Morozov and Babenko 2018).

Copyright © 2020, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

^{*}Co-first authors are ranked alphabetically. Correspondence to Xiao Yan.

¹Recall-time performance measures the time taken to reach a given recall for query processing.

In spite of its excellent performance, there lacks a good understanding why ip-NSW works well for MIPS. Morozov and Babenko (2018) proved that a greedy walk in the proximity graph will find the exact MIPS of a query if the graph is the Delaunay graph for inner product. Nevertheless, the ip-NSW graph is only an approximation of the Delaunay graph, which contains much more edges than the ip-NSW graph. It is not clear how accurately the ip-NSW graph approximates the Delaunay graph and how the quality of the approximation affects the performance of ip-NSW. Moreover, their theory does not provide insights on how to improve the performance of ip-NSW. For proximity graph based similarity search algorithms, a rigorous theoretical justification is usually difficult due to the complexity of real datasets. In this case, an intuitive explanation is helpful if it leads to a better understanding of the algorithm and provides insights for performance improvements.

Contributions. We make three main contributions in this paper. Firstly, we identify an important property of the MIPS problem — strong norm bias, which means large norm items are much more likely to be the result of MIPS. Although it is common sense that MIPS is biased towards large norm items, the interesting thing is the intensity of the norm bias we observed. In the four datasets we experimented, items ranking top 5% in norm occupy at least 87.5% and as high as 100% of the top-10 MIPS result. We also found that a skewed norm distribution, in which some items have much larger norm than others, is not a must for the strong norm bias to appear, and the large cardinality of modern datasets is also an important reason behind the strong norm bias.

Secondly, we explain the excellent performance of ip-NSW as matching the norm bias of the MIPS problem. We found that items with large norm have much higher in-degree than the average in the proximity graph built by ip-NSW and a graph walk spends a dominant portion of its computation on these items. Therefore, ip-NSW performs well for MIPS because it effectively avoids unnecessary computation on small-norm items, which are unlikely to be the results of MIPS.

Thirdly and most importantly, we propose the ip-NSW+ algorithm, which significantly improves the performance of ip-NSW. We found that the norm bias in ip-NSW can harm the performance of MIPS by spending computation on many large norm items that do not have a good inner product with the query. To tackle this problem, we introduce an additional angular proximity graph in ip-NSW+ and utilize the fact that items pointing to similar direction are likely to share similar MIPS neighbors. By retrieving the MIPS neighbors of the angular neighbors of the query, ip-NSW+ avoids computation on both small norm items and large norm items that do not have a good inner product with the query. To our knowledge, ip-NSW+ is the first similarity search algorithm that uses two proximity graphs constructed from different similarity functions. Experimental results show that ip-NSW+ not only significantly outperforms ip-NSW but also provides more robust performance under different data distributions.

Table 1: Dataset statistics

DATASET	# ITEMS	# DIMENSIONS
YAHOO!MUSIC	136,736	300
WORDVECTOR	1,000,000	300
IMAGENET	2,340,373	150
TINY5M	5,000,000	384

2 Norm Bias in MIPS

In this section, we show that there exists strong norm bias in the MIPS problem. We also argue that large dataset cardinality also contributes to the norm bias.

To find out to what extent norm affects an item’s chance of being the result of MIPS, we conducted the following experiment. We used four datasets, i.e., Yahoo!Music, WordVector, ImageNet and Tiny5M. Some statistics of the datasets can be found in Table 1 and more details are given in Section 5. For each dataset, we found the exact top-10 MIPS² result of 1,000 randomly selected queries using linear scan, which gave us a result set containing 10,000 items (duplicate items exist as an item can be in the results of multiple queries). We also partitioned the items into groups according to their norm, e.g., items ranking top 5% in norm and items ranking top 20%-25% in norm. Finally, for items in each norm group, we calculated the percentage they occupy in the result set, which is plotted in Figure 1.

Figure 1 shows that items with large norm are much more likely to be the result of MIPS. Specifically, items ranking top 5% in norm take up 89.5%, 87.5%, 93.1% and 100% in the ground truth top-10 MIPS results for Yahoo!Music, WordVector, ImageNet and Tiny5M, respectively. One may conjecture that the norm bias is caused by skewed norm distribution, in which the top ranking items have much larger norm than the others. We plot the norm distribution of the datasets in Figure 2 and it shows that this conjecture does not hold for Yahoo!Music and Tiny5M, in which most items have a norm close to the maximum. In fact, the 95% percentile³ of the norm distribution is only 1.16 times of the median norm for Yahoo!Music (1.37 for Tiny5M). Theorem 1 also shows that skewed norm distribution alone is not enough to explain the strong norm bias we observed.

Theorem 1. *For two independent random vectors x and y in $\mathbb{R}^d$, the entries of x are independent and $x_i \sim \mathcal{N}(0, \alpha)$ for $i = 1, 2, \dots, d$ with $\alpha \geq 1$, the entries of y are also independent and $y_i \sim \mathcal{N}(0, 1)$ for $i = 1, 2, \dots, d$. For a query $q \in \mathbb{R}^d$, we have $\mathbb{P}[q \top x \geq q \top y \mid q \top x \geq 0, q \top y \geq 0] = \frac{2}{\sqrt{\pi^2 \alpha}} \int_0^{+\infty} e^{-\frac{a^2}{2\alpha}} \int_0^a e^{-\frac{b^2}{2}} db da$.*

The proof can be found in the supplementary material⁴. Intuitively, Theorem 1 quantifies how likely larger norm

²Choosing top-10 MIPS is not arbitrary as it is widely adopted in related works.

³We define η_t , the $t\%$ percentile of the norm distribution, as $t = \frac{|\{x \in \mathcal{X}, \|x\| \leq \eta_t\}|}{|\mathcal{X}|} \times 100$.

⁴See <https://arxiv.org/pdf/1909.13459.pdf> for the supplementary material

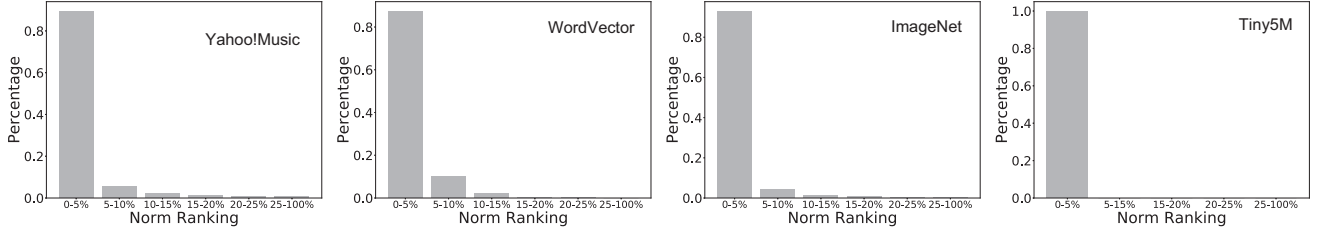


Figure 1: The percentage that items in each norm group occupy in the result set

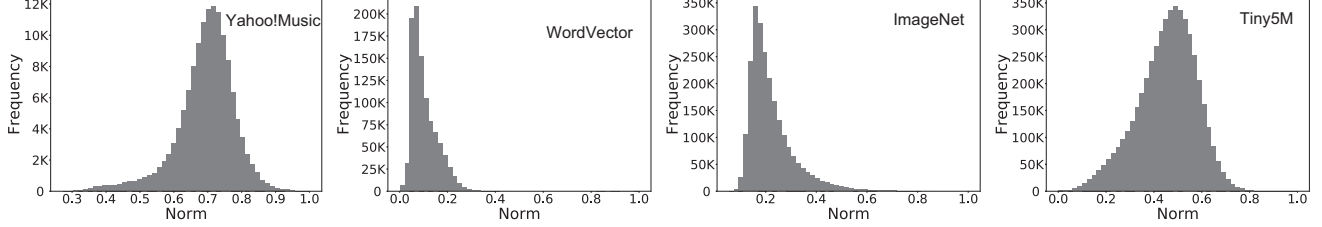


Figure 2: Norm distributions (maximum norm normalized to 1)

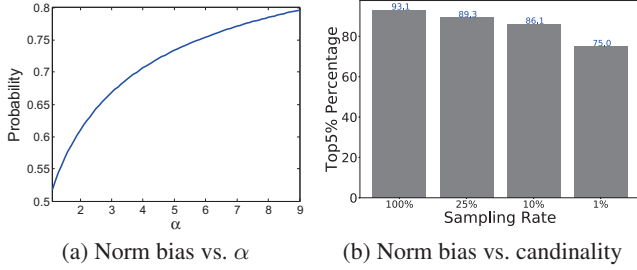


Figure 3: Analysis of the norm bias

will result in larger inner product. As $\mathbb{E}[\|x\|^2] = \alpha d$ and $\mathbb{E}[\|y\|^2] = d$, the norm of x is roughly $\sqrt{\alpha}$ times of y . We constrain the inner products to be non-negative because negative inner product is not interesting for many practical applications such as recommendation. $\mathbb{P}[q \top x \geq q \top y \mid q \top x \geq 0, q \top y \geq 0]$ is a function of α and we plot its curve in Figure 3a using numerical integration. The results show that larger norm only brings a modest probability (comparing with 0.5) of having larger inner product. For example, the probability of having larger inner product is only 0.56 with $\alpha = 1.35$. Recall that the 95% percentile norm is 1.16 times of the median for Yahoo!Music and $\sqrt{1.35} \approx 1.16$. However, the observed norm bias (items ranking top 5% in norm take up 89.5% of the top-10 MIPS result for Yahoo!Music) is much stronger than that is predicted by the norm distribution and this is also true for WordVector, ImageNet and Tiny5M.

We find that large dataset cardinality also contributes to the norm bias. Consider an item x with modest norm and there are m items having larger norm than x in the dataset. Item x only has a probability of $p = \prod_{i=1}^m p_i$ to be the MIPS of a query (if we assume all items are independent), in which $p_i = \mathbb{P}[q \top x \geq q \top z^i \mid q \top x \geq 0, q \top z^i \geq 0]$ and z^i is the

i -th item that has larger norm than x . As $p_i < 0.5$ and m is large for large datasets, the probability p is very small. This explanation suggests that the norm bias is stronger for larger datasets even if the norm distribution is the same. To validate, we uniformly sample the ImageNet dataset and plot the percentage that items ranking top 5% in norm occupy in the top-10 MIPS result in Figure 3b. Note that uniform sampling ensures that the shape of the norm distribution is the same across different sampling rate but a lower sampling rate results in smaller dataset cardinality. The results show that the top norm items take up a greater portion of the MIPS results under larger dataset cardinality, which validates our analysis. Our explanation justifies the extremely strong norm bias observed on the Tiny5M dataset even if its norm distribution is not skewed. Moreover, this explanation also implies that strong norm bias may be a universal phenomenon for modern datasets as they usually have large cardinality.

3 Understanding the Performance of ip-NSW

In this section, we briefly introduce the ip-NSW algorithm and show that ip-NSW has excellent performance because it matches the strong norm bias of the MIPS problem.

3.1 NSW

The query processing and index construction procedures of NSW are shown in Algorithm 1 and Algorithm 2, respectively. In Algorithm 1, a graph walk for a similarity search query q starts at an entry vertex v_0 (chosen randomly or deterministically) and keeps probing the neighbors of the unchecked vertex that is most similar to q in the candidate pool $\mathcal{C}$. The size of the candidate pool, l , controls the quality of the search results and the graph walk is more likely to get stuck at local optimum under small l ⁵.

⁵A graph walk with $l = 1$ is usually called greedy search.

For index construction, NSW does not require each item to connect to its exact top- M neighbors in the dataset. Items are inserted sequentially into the graph in Algorithm 2 and Algorithm 1 is used to find the approximate top- M neighbors for an item in the current graph. Therefore, constructing NSW is much more efficient than constructing an exact k-nearest neighbor graph (knn graph). ip-NSW builds and searches the graph using inner product $s(x, y) = x^\top y$ as the similarity function. We omit some details in Algorithm 1 and Algorithm 2 for conciseness, for example, ip-NSW actually adopts multiple hierarchical layers of NSW (known as HNSW) to improve performance. Readers may refer to (Malkov and Yashunin 2018) for more details.

Algorithm 1 NSW: Query Processing via Graph Walk (Fu et al. 2019)

```

1: Input: graph  $\mathcal{G}_s$ , similarity function  $s(x, y)$ , query  $q$ ,
   entry vertex  $v_0$ , candidate pool size  $l$ 
2: Initialize  $i = 0$ , candidate pool  $\mathcal{C} = \emptyset$  and  $\mathcal{C}.add(v_0)$ 
3: while  $i < l$  do
4:   Set  $v_{curr}$  as the first unchecked vertex in  $\mathcal{C}$  and set  $i$ 
     as its index in  $\mathcal{C}$ , mark  $v_{curr}$  as checked
5:   for every neighbour  $v$  of  $v_{curr}$  in  $\mathcal{G}_s$  do
6:     If  $v$  is not checked, calculate  $s(q, v)$  and  $\mathcal{C}.add(v)$ 
7:   Sort  $\mathcal{C}$  in descending order of  $s(q, v)$ 
8:   If  $\mathcal{C}.size() > l$ , execute  $\mathcal{C}.resize(l)$  by removing items
     with small  $s(q, v)$ 
9: return the top  $k$  vertexes in  $\mathcal{C}$ 

```

Algorithm 2 NSW: Graph Construction (Morozov and Babenko 2018)

```

1: Input: dataset  $\mathcal{X}$ , similarity function  $s(x, y)$ , maximum
   vertex degree  $M$ 
2: Initialize  $\mathcal{G}_s = \emptyset$ 
3: for each  $x$  in  $\mathcal{X}$  do
4:   Use Algorithm 1 to find  $M$  items most similar to  $x$ 
     w.r.t.  $s(x, y)$  in the current graph  $\mathcal{G}_s$ 
5:   Add  $x$  to  $\mathcal{G}_s$  by connecting it to the  $M$  items using
     directed edges
6: return  $\mathcal{G}_s$ 

```

3.2 Norm Bias in ip-NSW

We built ip-NSW graphs for the four datasets in Table 1 and plot the average in-degree for items in each norm group in Figure 4. The results show that the large norm items have much higher in-degrees than the average. To be more specific, the average in-degrees for items ranking top 5% in norm are 3.2, 8.0, 11.1 and 19.8 times of the dataset average for Yahoo!Music, WordVector, ImageNet and Tiny5M, respectively. This is not surprising as the large norm items are more likely to have large inner product with other items as shown in Section 2. The insertion based graph construction procedure of ip-NSW may also contribute to the skewed in-degree distribution. A new item builds its connections by

checking the neighbors of existing items and the initially inserted items are likely to connect to the large norm items, which means that graph construction tend to amplify the in-degree skewness. Having large in-degrees means that the large norm items are well-connected in the ip-NSW graph, which makes it more likely for a graph walk to reach them.

To better understand a walk in the ip-NSW graph, we conducted MIPS using ip-NSW for 1,000 randomly selected queries. We recorded the id of the item when inner product was computed, and plot the percentage of inner product computation conducted on items in each norm group in Figure 5. The results show that most of the inner product computation was conducted on the large norm items. For Yahoo!Music, WordVector, ImageNet and Tiny5M, items ranking top 5% in norm take up 80.7%, 93.1%, 88.6% and 100% of the inner product computation. Compared with the in-degree distributions in Figure 4, the computation distributions are even more biased towards the large norm items. This suggests that a walk in the ip-NSW graph reaches the large norm items very quickly and keeps moving among these items. With these results, we can conclude that ip-NSW is also biased towards the large norm items, in terms of both connectivity and computation. The norm bias of ip-NSW allows it to effectively avoid unnecessary computation on small norm items that are unlikely to be the result of MIPS. Therefore, ip-NSW has excellent performance mainly because it matches the strong norm bias of the MIPS problem.

4 The ip-NSW+ Algorithm

In this section, we present the ip-NSW+ algorithm, which is motivated by an analysis indicating that the norm bias of ip-NSW can lead to inefficient MIPS.

4.1 Motivation

We have shown in Section 3 that ip-NSW has a strong norm bias, which helps to avoid computation on small norm items. However, this norm bias can result in inefficient MIPS and we illustrate this point with an example in Figure 6a, in which y is an MIPS neighbor of x and z is an MIPS neighbor of y . As y and z are the MIPS neighbors of some item, they usually have large norm due to the norm bias of the MIPS problems but the angles (φ and ω) are not necessarily small, especially when the norm of y and z are very large. Suppose that x is the query and the graph walk is now at y , in the next step, the graph walk will compute $x^\top z$ but z may not have a good inner product with x due to the large angle (i.e., $\varphi + \omega$) between them. This example shows that ip-NSW may spend computation on many large norm items that do not have a good inner product with the query because the large norm items are well connected in the ip-NSW graph.

The problem of ip-NSW is caused by the rule it adopts — the MIPS neighbor of an MIPS neighbor is also likely to be an MIPS neighbor, which is not necessarily true. To improve ip-NSW, we need a new rule that satisfies two requirements. First, it should match the norm bias of the MIPS problems and avoid computation on small norm items, which ip-NSW does well. Second, it should also avoid computation on large

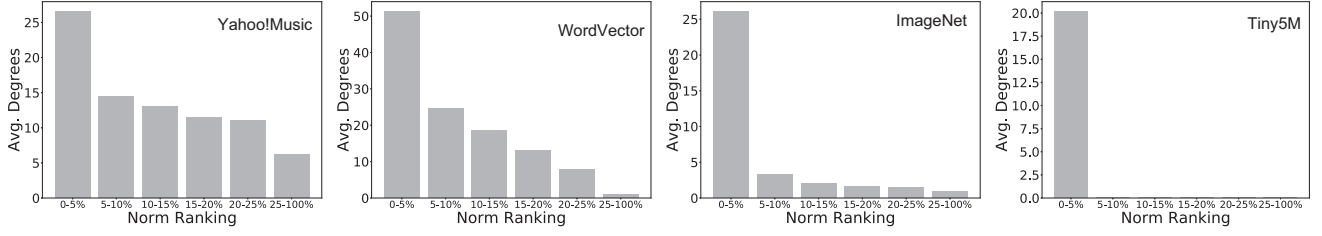


Figure 4: The average in-degree distribution for items in each norm group

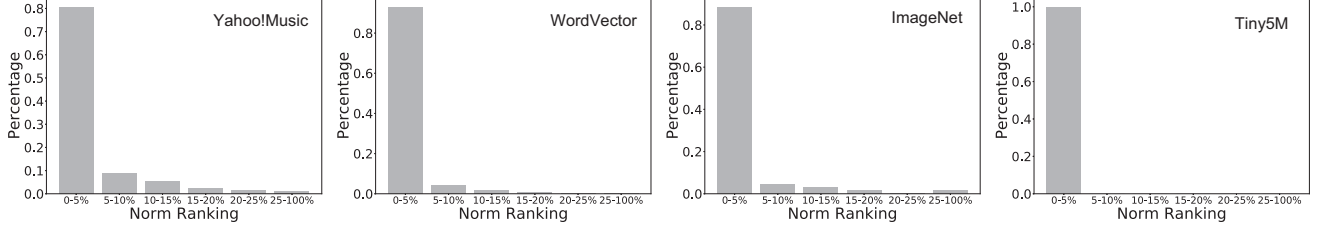


Figure 5: The percentage of inner product computation conducted on items in each norm group

norm items that do not have a good inner product with the query, which is the main problem of ip-NSW.

We propose an alternative rule — the MIPS neighbor of an angular neighbor is likely to be an MIPS neighbor, which satisfies the two requirements. We define the angular similarity between two vector x and y as $s_a(x, y) = \frac{x^\top y}{\|x\| \|y\|}$ and say that y is an angular neighbor of x if $s_a(x, y)$ is large. Specifically, this rule says that for a query x and its angular neighbor y in a dataset, if z is an MIPS neighbor of y in the dataset, then $x^\top z$ is likely to be large. We provide an illustration of this rule in Figure 6b. In the figure, z is an MIPS neighbor of y , thus z usually has large norm, meeting the first requirement. The angle $\varphi + \omega$ is usually not too large as y is an angular neighbor of x and φ is small, and thus $x^\top z$ is likely to be large, meeting the second requirement. Theorem 2 formally establishes that an MIPS neighbor of an angular neighbor is a good MIPS neighbor with an assumption about z .

Theorem 2. For two vectors x and y in $\mathbb{R}^d$ having an angular similarity $\beta = \frac{x^\top y}{\|x\| \|y\|}$, a third vector $z \in \mathbb{R}^d$ and the entries of z are independent and $z_i \sim \mathcal{N}(0, 1)$ for $i = 1, 2, \dots, d$, given $y^\top z = \gamma$, we have $x^\top z \mid y^\top z = \gamma \sim \mathcal{N}(\frac{\gamma\beta\|x\|}{\|y\|}, \|x\|^2(1 - \beta^2))$.

The proof can be found in the supplementary material. If x is a query and y is the angular neighbor of x in the dataset, which means that $\beta = \frac{x^\top y}{\|x\| \|y\|}$ is large. If y and z are both in the dataset and z is an MIPS neighbor of y , we have $y^\top z = \rho\|y\|\|z\|$, in which ρ is large. Given these conditions and using Theorem 2, we have $x^\top z \mid y^\top z = \rho\|y\|\|z\| \sim \mathcal{N}(\rho\beta\|x\|\|z\|, \|x\|^2(1 - \beta^2))$, which means the inner product between x and z follows a Gaussian distribution. The mean of the distribution ($\rho\beta\|x\|\|z\|$) is large as both ρ and β are large. Moreover, the variance of the distribution ($\|x\|^2(1 - \beta^2)$) is small as β is large. Therefore, there

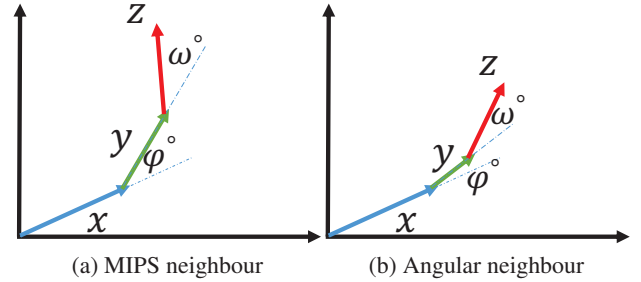


Figure 6: Example of MIPS neighbor and angular neighbor

is a good chance that $x^\top z$ is a large.

Theorem 2 is also supported by empirical results from the following experiment. We conducted search for 1,000 randomly selected queries on Yahoo!Music and ImageNet. For each query, we find its ground truth top-10 angular neighbors in the dataset and for each of these angular neighbors, we find its ground truth top-10 MIPS neighbors in the dataset. This procedure gives us a result set containing 100 candidates (with possible duplication) for each query, which can be used to calculate the recall for top-10 MIPS. The average recalls were 82.67% and 97.22% for Yahoo!Music and ImageNet, respectively, which suggests that aggregating the MIPS neighbors of the angular neighbors can obtain a good recall for MIPS. In contrast, aggregating the MIPS neighbors of the ground-truth top-10 MIPS neighbors of a query only provide a recall of 67.21% for ImageNet.

4.2 ip-NSW+

Based on the new rule presented in Section 4.1, we present the query processing procedure of ip-NSW+ in Algorithm 3.

To find the angular neighbors of the query, ip-NSW+ searches an angular NSW graph $\mathcal{A}_s$ because NSW provides

Algorithm 3 ip-NSW+: Query Processing via Graph Walk

- 1: **Input:** an angular NSW graph $\mathcal{A}_s$, an inner product NSW graph $\mathcal{G}_s$, query q , starting vertex v_0 in $\mathcal{A}_s$
 - 2: Conduct search on $\mathcal{A}_s$ using Algorithm 1 to find the top- k' angular neighbors of q
 - 3: **for** each item v in the top- k' angular neighbors **do**
 - 4: **for** each edge (v, u) in the ip-NSW graph $\mathcal{G}_s$ **do**
 - 5: $\mathcal{C}.add(u)$
 - 6: Conduct search on $\mathcal{G}_s$ with $\mathcal{C}$ using Algorithm 1 to find the top- k inner product neighbors of q
-

excellent performance on many similarity search problems. Instead of finding the exact inner product neighbors of the angular search results, ip-NSW+ uses their neighbors in the inner product graph $\mathcal{G}_s$ as an approximation. After the initialization (line 2-5 in Algorithm 3), the candidate queue $\mathcal{C}$ already contains a good portion of the MIPS result for the query and the time spent to find them by graph walk on ip-NSW can be saved. To further refine the result in $\mathcal{C}$, a standard graph walk on the inner product graph $\mathcal{G}_s$ is conducted in line 6 of Algorithm 3.

For index construction, ip-NSW+ builds $\mathcal{A}_s$ and $\mathcal{G}_s$ simultaneously and the items are inserted sequentially (in a random order) into the two graphs. For an item v , it is first inserted into $\mathcal{A}_s$ with Algorithm 2 using angular similarity as the similarity function. Then, v is inserted into $\mathcal{G}_s$ and the neighbors of v in $\mathcal{G}_s$ are found using ip-NSW+ (Algorithm 3) instead of ip-NSW (Algorithm 1). Empirically, we found that this provides more accurate inner product neighbors for the items and hence better search performance. One subtlety of ip-NSW+ is controlling the time spent on angular neighbor search (ANS). Spending too much time for ANS means only a short time is left for result refinement by a graph walk on the inner product graph $\mathcal{G}_s$, which harms performance. As the time consumption of a graph walk in NSW is controlled by the maximum degree M (the complexity of each step) and the candidate pool size l (how many steps will be taken), we use smaller M and l for the angular graph $\mathcal{A}_s$ than for the inner product graph $\mathcal{G}_s$. We show in Section 5 that ip-NSW+ using fixed M and l without dataset-specific tuning already performs significantly better than ip-NSW.

The index construction complexity of ip-NSW+ is approximately twice of ip-NSW as ip-NSW+ constructs two proximity graphs. The index size of ip-NSW+ is less than twice of ip-NSW because we use small M for the angular graph $\mathcal{A}_s$. These additional complexities will not be a big problem because the insertion-based graph construction of NSW is efficient and the memory of a single machine is sufficient for most datasets. However, ip-NSW+ provides significantly better recall-time performance than ip-NSW (see Section 5), which benefits many applications. Existing proximity-graph-based algorithms use a single proximity graph and the same similarity function is used for both index construction and query processing. In contrast, ip-NSW+ uses two proximity graphs constructed from different similarity functions jointly, which is a new paradigm for proximity-graph-based similarity search and may inspire fu-

ture research.

5 Experimental Results

Datasets and settings. We used the four datasets listed in Table 1. Yahoo!Music is obtained by conducting ALS-based matrix factorization (Yun et al. 2013) on the user-item ratings in the Yahoo!Music dataset. We used the item embeddings as dataset items and the user embeddings as queries. WordVector is sampled from the word2vec embeddings released in (Mikolov et al. 2013), and ImageNet contains the visual descriptors of the ImageNet images (Deng et al. 2009). Tiny5M is sampled from the Tiny80M dataset and contains visual descriptors of the Tiny images. Unless otherwise stated, we test the performance of top-10 MIPS and use recall as the performance metric. For top- k MIPS, an algorithm only returns the k best items it finds. Denote the results an algorithm returns for a query as $\mathbb{S}'$ and the ground truth top- k MIPS of the query as $\mathbb{S}$, recall is defined as $r = |\mathbb{S}' \cap \mathbb{S}|/|\mathbb{S}'|$. We report the average recall of 10,000 randomly selected queries. We used $M = 10$ and $l = 10$ for the angular graph $\mathcal{A}_s$ in ip-NSW+ in all experiments and the parameter configurations of $\mathcal{G}_s$ in ip-NSW+ is the same as the inner product graph in ip-NSW. The experiments were conducted on a machine with Intel Xeon E5-2620 CPU and 48 GB memory in a single-thread mode. For ip-NSW+, the reported time includes searching both the angular graph $\mathcal{A}_s$ and the inner product graph $\mathcal{G}_s$. We implemented ip-NSW+ by modifying the code of ip-NSW and did not introduce extra optimizations to make ip-NSW+ run faster ⁶.

Direct comparison. We report the recall-time performance of ip-NSW and ip-NSW+ in Figure 7. We also tested Simple-LSH (Neyshabur and Srebro 2015), the state-of-the-art LSH-based method for MIPS. We used the implementation provided in (Yu et al. 2017) and tuned the parameters following (Morozov and Babenko 2018). However, the performance of Simple-LSH is significantly poorer and plotting its recall-time curve in Figure 7 will make the figure hard to read, and thus we report its curve in the supplementary material. As an example, Simple-LSH takes 598ms to reach a recall of 0.732 for WordVector and 1035ms to reach a recall of 0.735 for ImageNet. This is actually worse than the exact MIPS method, FEXIPRO (Li et al. 2017), which uses multiple pruning rules to speed up linear scan, and takes 20.9ms, 196.3ms and 179.5ms on average for each query on Yahoo!Music, WordVector and ImageNet, respectively ⁷. FEXIPRO, however, is at least an order of magnitude slower than ip-NSW and ip-NSW+, as shown in Figure 7, which confirms the results in (Morozov and Babenko 2018) that ip-NSW outperforms existing algorithms. Importantly, ip-NSW+ is able to further make significant improvements over ip-NSW. For example, ip-NSW+ reaches a recall of 0.9 at a speed that is 11 times faster than ip-NSW (0.5 ms vs 5.5 ms) on the ImageNet dataset. Even on the Tiny5M dataset, which has the strongest norm bias and items ranking top 5%

⁶See <https://github.com/jerry-liujie/ip-nsw/tree/GraphMIPS> for all experiment code and data.

⁷We did not provide the performance for FEXIPRO on Tiny5M as it goes out of memory.

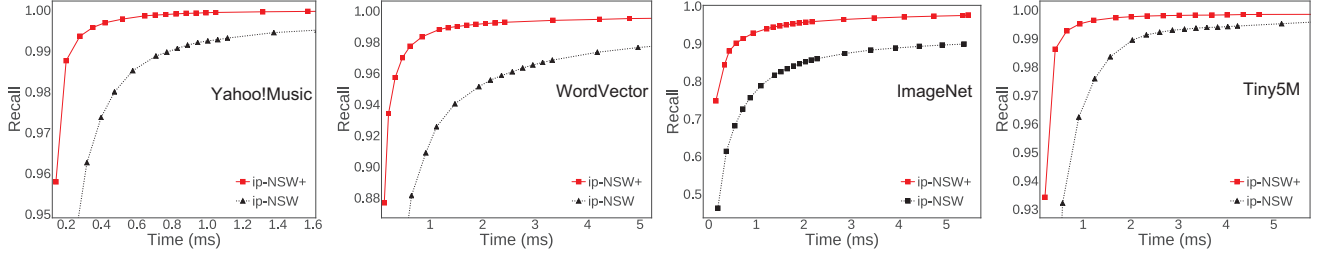


Figure 7: Recall-time performance comparison between ip-NSW and ip-NSW+

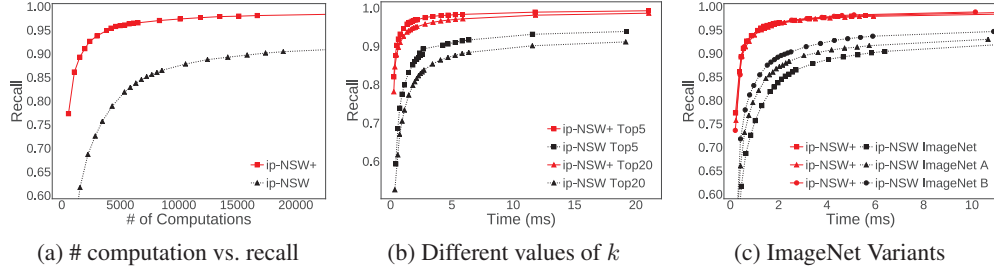


Figure 8: Additional experimental results on the ImageNet dataset (best viewed in color)

in norm occupy 100% of top-10 MIPS result, ip-NSW+ still outperforms ip-NSW.

More experiments. We conducted this set of experiments on the ImageNet dataset to further examine the performance of ip-NSW+. In Figure 8a, we compare the recall of ip-NSW and ip-NSW+ with respect to the number of similarity function evaluations since similarity function evaluation is usually the most time-consuming part of an algorithm. We count as one similarity function evaluation when ip-NSW computes inner product with one item and ip-NSW+ computes angular similarity or inner product with one item. The results show that ip-NSW+ spends much less computation than ip-NSW for the same recall, suggesting the performance gain of ip-NSW+ indeed comes from a better algorithm design. In Figure 8b, we compare ip-NSW and ip-NSW+ for top-5 MIPS and top-20 MIPS, which shows that ip-NSW+ consistently outperforms ip-NSW for different k .

One surprising phenomenon is that ip-NSW+ provides more robust performance than ip-NSW under different transformations of the norm distribution. We created two variants of the ImageNet dataset, i.e., ImageNet-A and ImageNet-B, by scaling the items without changing their directions. ImageNet-A and ImageNet-B add 0.18 and 0.36 to the Euclidean norm of each item, respectively. The norm distributions of the transformed datasets can be found in the supplementary material. We define the tailing factor (TF) of a dataset as the ratio between the 95% percentile of the norm distribution and the median norm and say that the norm distribution is more skewed when the TF is large. The TFs of ImageNet, ImageNet-A and ImageNet-B are 2.05, 1.55 and 1.37, respectively. We report the performance of ip-NSW and ip-NSW+ on the three datasets in Figure 8c. The results show that ip-NSW+ has almost identical performance on the three ImageNet variants and consistently outperforms

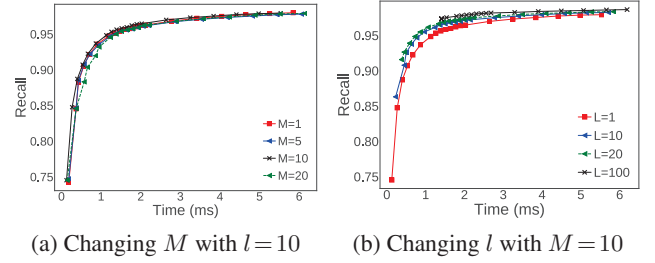


Figure 9: Performance under different M and l on ImageNet

ip-NSW. In contrast, the performance of ip-NSW varies a lot, the best performance is achieved on ImageNet-B (with the smallest TF) while the worst performance is observed on ImageNet (with the largest TF). We tried more datasets and another method to scale the items in the supplementary material and the results show that ip-NSW+ consistently provides more robust performance than ip-NSW. Moreover, ip-NSW usually performs better when the TF is small.

We try to explain this phenomenon as follows. The norm bias in ip-NSW is more severe when the norm distribution is more skewed. Therefore, ip-NSW will compute inner product with more large norm items that do not have a good inner product with the query and hence its performance worsens. In contrast, ip-NSW+ collects the MIPS neighbors of the angular neighbors and these neighbors are shown to have a good inner product with the query in Theorem 2. The stable performance of ip-NSW indicates that it effectively avoids computing inner product with items having large norm but not likely to be the results of MIPS.

We report the performance of ip-NSW+ when using different configurations of M and l for the angular graph $\mathcal{A}_s$ in

Figure 9. Both M and l control the time spent on searching the angular neighbors, and setting a large value for them produces more accurate angular neighbors at the cost of using more time. The results show that the performance gap between different configurations is small and hence ip-NSW+ is robust to the choice of M and l . The curves in Figure 9b start at different time because the time spent on angular neighbor search varies a lot under different l (but not the case for M), and ip-NSW+ produces MIPS results only after switching to the inner product graph. The results also show that using both too small values (e.g., $M = 1$ or $l = 1$, producing angular neighbors of poor quality) and too large values (e.g., $M = 20$ or $l = 100$, spending too much time for angular neighbor search) for the two parameters degrades performance. Therefore, we recommend to use the default setting with $M = 10$ and $l = 10$.

6 Conclusions

In this paper, we identified an interesting phenomenon for the MIPS problem — norm bias, which means that large norm items are much more likely to be the results of MIPS. We showed that ip-NSW achieves excellent performance for MIPS because it also has a strong norm bias, which means that the large norm items have large in-degrees in the ip-NSW graph and the majority of computation is conducted on them. We also proposed the ip-NSW+ algorithm, which avoids computation on large norm items that do not have a good inner product with the query. Experimental results show that ip-NSW+ significantly outperforms ip-NSW and is more robust to different data distributions.

Acknowledgments. This work was supported by ITF 6904945, and GRF 14208318 & 14222816, and the National Natural Science Foundation of China (NSFC) (Grant No. 61672552).

References

- Abuzaid, F.; Sethi, G.; Bailis, P.; and Zaharia, M. 2019. To index or not to index: Optimizing exact maximum inner product search. In *ICDE*, 1250–1261.
- Bachrach, Y.; Finkelstein, Y.; Gilad-Bachrach, R.; Katzir, L.; Koenigstein, N.; Nice, N.; and Paquet, U. 2014. Speeding up the xbox recommender system using a euclidean transformation for inner-product spaces. In *Recsys*, 257–264. ACM.
- Chandar, S.; Ahn, S.; Larochelle, H.; Vincent, P.; Tesauro, G.; and Bengio, Y. 2016. Hierarchical memory networks. *arXiv preprint arXiv:1605.07427*.
- Deng, J.; Dong, W.; Socher, R.; Li, L. J.; Li, K.; and Fei-Fei, L. 2009. Imagenet: A large-scale hierarchical image database. In *CVPR*, 248–255.
- Felzenszwalb, P. F.; Girshick, R. B.; McAllester, D. A.; and Ramanan, D. 2010. Object detection with discriminatively trained part-based models. *IEEE TPAMI* 32:1627–1645.
- Fu, C.; Xiang, C.; Wang, C.; and Cai, D. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Vldb* 12(5):461–474.
- Hajebi, K.; Abbasi-Yadkori, Y.; Shahbazi, H.; and Zhang, H. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *IJCAI*, 1312–1317.
- Harwood, B., and Drummond, T. 2016. Fanng: Fast approximate nearest neighbour graphs. In *CVPR*, 5713–5722.
- Jun, K.-S.; Bhargava, A.; Nowak, R.; and Willett, R. 2017. Scalable generalized linear bandits: Online computation and hashing. In *NeurIPS*, 99–109.
- Koren, Y.; Bell, R. M.; and Volinsky, C. 2009. Matrix factorization techniques for recommender systems. *IEEE Computer* 42:30–37.
- Li, H.; Chan, T. N.; Yiu, M. L.; and Mamoulis, N. 2017. FEXIPRO: fast and exact inner product retrieval in recommender systems. In *SIGMOD*, 835–850.
- Li, J.; Yan, X.; Zhang, J.; Xu, A.; Cheng, J.; Liu, J.; Ng, K. K.; and Cheng, T. C. 2018. A general and efficient querying method for learning to hash. In *SIGMOD*, 1333–1347.
- Malkov, Y. A., and Yashunin, D. A. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*.
- Malkov, Y.; Ponomarenko, A.; Logvinov, A.; and Krylov, V. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45:61–68.
- Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G. S.; and Dean, J. 2013. Distributed representations of words and phrases and their compositionality. In *NeurIPS*, 3111–3119.
- Morozov, S., and Babenko, A. 2018. Non-metric similarity graphs for maximum inner product search. In *NeurIPS*, 4726–4735.
- Neyshabur, B., and Srebro, N. 2015. On symmetric and asymmetric lshs for inner product search. In *ICML*, 1926–1934.
- Ram, P., and Gray, A. G. 2012. Maximum inner-product search using cone trees. In *SIGKDD*, 931–939.
- Shrivastava, A., and Li, P. 2014. Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). In *NeurIPS*, 2321–2329.
- Teflioudi, C.; Gemulla, R.; and Mykytiuk, O. 2015. LEMP: fast retrieval of large entries in a matrix product. In *SIGMOD*, 107–122.
- Wang, J.; Wang, J.; Zeng, G.; Gan, R.; Li, S.; and Guo, B. 2013. Fast neighborhood graph search using cartesian concatenation. In *ICCV*, 2128–2135.
- Yan, X.; Li, J.; Dai, X.; Chen, H.; and Cheng, J. 2018. Norm-ranging LSH for maximum inner product search. In *NeurIPS*, 2956–2965.
- Yu, H.-F.; Hsieh, C.-J.; Lei, Q.; and Dhillon, I. S. 2017. A greedy approach for budgeted maximum inner product search. In *NeurIPS*, 5453–5462.
- Yun, H.; Yu, H. F.; Hsieh, C.; Vishwanathan, S. V. N.; and Dhillon, I. S. 2013. NOMAD: non-locking, stochastic multi-machine algorithm for asynchronous and decentralized matrix completion. *CoRR*.